

Homework Assignment #3

Professor: Eric Gerber

Name: _____

Instructions: Please include the following information on the first page of your completed homework write-up:

1. Your name
2. DS 4420
3. Homework #3

You will submit **up to four files** to Gradescope for this homework:

- Handwritten/latex typeset answers may be included in their own **.pdf** file or as part of either of the below
- A **.ipynb** file with all python code/output
- A **.pdf** file knitted from an included **.rmd** with all R code/output

Answers that are not supported by reasoning/work will not receive full credit. **Homework is due by 11:59 pm, via Gradescope, on the date above.** Late submissions will **not** be accepted, but you may receive extra credit for early submission (see syllabus for details).

You will also be graded on organization/neatness of the submitted files.

User-User Collaborative Filtering (40 points)

(1: 10 pts) Apply user-user collaborative filtering **with the cosine similarity score** to predict **all** the missing ratings (on a scale of 1-10) given the following data table. Use the $k = 2$ most similar users to predict. For the **first two** missing ratings ($r(U_1, I_1)$ and $r(U_2, I_2)$) show **complete** work **BY HAND**, then you may find the rest using your function from the next part (from (2)). **Note:** you should always Min-Max scale similarity scores.

	Item 1	Item 2	Item 3	Item 4	Item 5	Item 6
User 1		5	3	10	7	8
User 2	7		6	4		1
User 3	5	3		8	3	10
User 4	6	9	6		1	3

(2: 30 pts) On Canvas, there is the **fake_rmp.csv** file which is a simulated data set of 100 students and their ratings of 15 professors (on a scale of 1-5). Read the data into python making sure that the first column is treated as the index:

```
rmp_df = pd.read_csv('fake_rmp.csv', index_col = 0)
rmp_df.head()
```

(a) In **python**, write a function called **user_collab_filter()** which:

- Takes as arguments a pandas data frame (such as the **fake_rmp.csv** file), the name of a target user (the user id, e.g. "Student_1"), a string which specifies which similarity metric (**L2** or **Cosine**) to use, and a value **k** which specifies the number of most similar users to predict the target user with.

- Performs user-based collaborative filtering according to the strategy covered in the slides (Lecture 4, Slide 25) given the specified arguments and outputs the predicted ratings of the professors for the target user.

– **Notes and Hints:**

- * By specifying the target user at the outset, it should simplify the computation time; your function should only need to calculate similarity scores for the target user with all other users (rather than calculating all pairwise similarity scores)
- * Use centered and imputed data to compute similarities, but produce predictions on the original rating scale.
- * For the differences between L2 and Cosine:
 - After calculating all the similarity scores, you should **MinMax Scale** them, so that the weighted average prediction works for both similarity metrics (L2 or **Cosine**).
 - **BUT** before scaling, you'll want to take the negative of the L2 norm, so that smaller (more negative) values are less similar (and will map to 0).
- * The `.nlargest()` function may be helpful for identifying the k largest similarity scores.
- * Some students will not have missing ratings. You should include a check for this in the function which exits the function and gives a message pointing this out before running the algorithm.
- * You probably also want a check to see if your target user is even in the data. If the user names are the matrix rows, something like:

```
if target_user not in data.index:
    print(f"Error: {target_user} not found in the dataset.")
    return None
```

(b) Test that your function works by verifying:

- "Student_5", with $k = 3$ and using Cosine similarity, has a predicted value of approximately (within ± 1 of) 4.15 for Dr. Johnson.
- "Student_3" returns the message that they have no missing ratings.

(c) Students 1, 2, and 13 are all considering taking Dr. Gerber (the first professor in the data) next Fall (they have not taken him yet). Choose some value of $k > 4$. Use your function to predict what their ratings would be under **both the L2 and Cosine similarity metrics**, and then decide if there is consensus for each student and, if so, would you recommend that they take Dr. Gerber.

(For 15 pts. extra credit) Repeat parts (a)-(c) in R (5 points for each part successfully replicated in R).

Item-Item Collaborative Filtering (45 points)

(3: 15 pts) Given the table from problem (1):

(a) Apply item-item collaborative filtering **with the cosine similarity score** to predict **all** the missing ratings (on a scale of 1-10) using the $k = 2$ most similar items. For the **first two** missing ratings ($r(U_1, I_1)$ and $r(U_2, I_2)$) show **complete** work **BY HAND**, then you may find the rest using your function from the next part (from (4)). **Note:** you should always Min-Max scale similarity scores.

(b) **Discuss** in a few sentences how you would go about determining which algorithm (User-User or Item-Item) is more accurate/produces better predictions.

THE HOMEWORK CONTINUES ON THE NEXT PAGE

(4: 30 pts)

(a) In python create the `collab_filter()` function which updates the function from problem (2) by adding an additional argument `type` which can be either "User" or "Item", where "User" runs the user-user collaborative filtering algorithm from (2), while "Item" runs the item-item collaborative filtering algorithm (as defined in Lecture 4, Slide 40). Recall that for item-item CF:

- We center the data based on the items (so you are calculating the mean of the Professors ratings now and centering by column instead of by row).
- Compute similarity using only users who rated both items (co-rated set). In other words: we are calculating pairwise similarity scores for **all items/professors**, while **ignoring any users** who have not rated them. You might want to write a helper function that does this to avoid your `collab_filter()` function from getting too messy...
 - Do *not* fill missing values with 0 for item–item similarity; use only co-rated entries.
- When calculating the weighted similarity scores, if the target user has not rated one of the most similar items, simply fill that missing value with the similar item's mean.

(b) Test that your function works for item-item CF by verifying that "Student_5", with $k = 2$ and using Cosine similarity with `type = "Item"`, has a predicted value of approximately (within ± 1 of) 3.54 for Dr. Johnson.

(c) Repeat problem (2: c) but with item-item collaborative filtering and discuss if there are any differences.

CF Problem Ideation (15 points)

(5) Discuss a real data problem and identify a corresponding data set, which you think would be appropriate to analyze with Collaborative Filtering (or hybrid CF and Content-Based Filtering). You may think of using Collaborative Filtering (either user-user, item-item, or matrix factorization) in either recommendation system or non-recommendation system settings (you may want to discuss with the TAs or Dr. Gerber how that might work). The data set **must not** be a standard CF dataset (e.g., MovieLens, Netflix, Goodreads: the TAs and Dr. Gerber will be checking). Provide:

(a) A working link to the data set or to the website(s) from which the data set may be collected and a description of the problem of interest and how the data could be used to answer the problem with Collaborative Filtering.

(b) A discussion of any ethical concerns there might be in using the data and/or using the results of the model.

(c) Your intuition as to why Collaborative Filtering would be appropriate for this problem.

(Extra Credit: 5 points) Read through Simon Funk's blog posts detailing his approach to a model-based recommender system using Singular Value Decomposition (linked in the slides, but also here: ([blog post 1](#)), ([blog post 2](#)), ([blog post 3](#))). For reference, you can also check out packages that implement the Funk MF algorithm in [R](#) and in [Python](#).

Write a paragraph or two discussing in detail what lessons from our coverage of recommender systems persist in his approach, what aspects are new/more advanced (especially any implementation details from the R and Python packages you notice). In your discussion, you might consider issues such as similarity vs latent factors, handling missing data, scalability, and the role of optimization.